

Examen Pregunta 1

Una empresa de logística registra cronológicamente en un archivo digital cada entrega realizada por su flota de repartidores por cuenta de sus clientes. Cada registro contiene: `fecha_entrega`, `hora_entrega`, `rut_cliente`, `id_repartidor`, `id_destino`, `duración_entrega`. Se desea procesar el registro señalado para enviar a cada cliente el listado con los destinos y duraciones de las entregas realizadas por cada repartidor, manteniendo el orden cronológico en que cada repartidor efectuó sus entregas.

- a) [2,0 puntos] Proponga una solución (algoritmo) para obtener la información indicada de la manera más eficiente posible en tiempo.

Puntaje	Pauta
1,5	Ordenar AD por <code>rut_cliente</code> utilizando <code>radixSort()</code> , utilizando <code>CountingSort</code> como algoritmo estable. Luego AD queda ordenado por <code>rut_cliente</code> y para cada <code>rut_cliente</code> los registros están ordenados cronológicamente
0,5	<code>RadixSort()</code> tiene complejidad de tiempo $O(n)$, que es la estrategia de orden más eficiente de las vistas en clases.

- b) [2,0 puntos] Muestre que su solución cumple los requisitos de agrupación por cliente y de preservación del orden cronológico de las entregas dentro de cada grupo.

Puntaje	Pauta
0,5	Reconoce que el archivo digital (AD) inicialmente está ordenado cronológicamente
1,0	Al ordenar AD por <code>rut_cliente</code> utilizando <code>radixSort()</code> , y utilizando <code>CountingSort</code> como algoritmo estable, el ordenamiento es estable, luego se mantiene el orden cronológico de los registros asociados a cada <code>rut_cliente</code>
0,5	Luego para “enviar al cliente” el listado basta recorrer AD para cada <code>rut_cliente</code> .

- c) [2,0 puntos] Un compañero propone aplicar *QuickSort* sobre el registro completo, ordenándolo por `rut_cliente`, y luego separar los registros para formar los grupos. Compare su propuesta con la de su compañero en términos de eficiencia y correctitud.

Puntaje	Pauta
---------	-------

1,0	QuickSort() no es estable, luego sería necesario después de ordenar por rut_cliente ordenar cada grupo (rut iguales) cronológicamente para lograr el resultado esperado.
1,0	QuickSort tiene complejidad $O(n \log n)$. Luego para resolver correctamente, si m es el número de rut_cliente distintos, la complejidad sería $m * O(n \log n)$, lo cual es menos eficiente que RadixSort() que es $O(n)$

Examen Pregunta 2

La USS Enterprise viaja por el espacio a velocidad mayor que la de la luz mediante su motor Warp. Para generar un campo Warp estable, el reactor materia-antimateria es alimentado con cristales de Dilitio. Para alimentar eficientemente el motor Warp la nave cuenta con N líneas de inyección de Dilitio L_i , $i = 0..N-1$. Cada línea almacena M cristales y los entrega, en el orden en que fueron almacenados, al sistema de alimentación del reactor. El sistema de alimentación tiene N posiciones A_i , $i = 0..N-1$, donde cada posición A_i está asociada exclusivamente a la línea de inyección L_i .

Inicialmente, cada posición A_i contiene un cristal de Dilitio proveniente de su línea asociada. Durante cada ciclo i de operación del motor Warp, el reactor consume secuencialmente el cristal ubicado la posición A_i . Al consumir el cristal de la posición A_i , esta queda libre. Inmediatamente, el sistema de inyección toma el siguiente cristal de la línea L_i y lo instala en la posición libre A_i , de modo que el sistema de alimentación esté completo antes del siguiente ciclo de operación del reactor. Nota: El ciclo de operación del motor Warp hace variar i secuencialmente desde 0 a $N-1$ y luego a 0 nuevamente.

- a) [1,0 punto] Defina las estructuras de datos apropiadas para modelar el sistema actual de inyección y alimentación de cristales de Dilitio al reactor Warp.

Puntaje	Pauta
1,0	Las Líneas de Inyección L_i se comportan como una cola FIFO. Menos 0,5 si solo indican que es una lista o arreglo. El sistema de Alimentación A es un Arreglo de tamaño N . Menos 0,5 si lo modelan como una lista.

- b) [2,0 puntos] Escriba el pseudocódigo para la función *getCristal(i , L , A)* que opera y obtiene desde las líneas de inyección y alimentación el cristal para alimentar al reactor en cada ciclo de operación.

Puntaje	Pauta
1,0	A partir del valor de i , extrae $A[i]$ y lo alimenta al motor
1,0	Extrae de la cola FIFO $L[i]$ el primer elemento: $L[i].pop()$; Y lo ubica en $A[i]$

- c) [2,0 puntos] La producción de cristales de Dilitio es un proceso rápido y de bajo costo, pero con una alta variabilidad en la calidad C de cada cristal, la cual se mide y registra en cada cristal. La calidad de los cristales es relevante para la estabilidad y eficiencia del motor Warp. El ingeniero Scott decide mejorar la calidad global de los cristales

que alimentan al reactor Warp, modificando lo menos posible la configuración actual: cada posición A_i debe seguir siendo abastecida exclusivamente por su línea L_i . Proponga a Scotty una modificación del sistema que permita alimentar al reactor en cada ciclo el cristal de mayor calidad disponible.

Puntaje	Pauta
1,5	Modelar A como un max HEAP basado en la prioridad de los cristales
0,5	Modelar cada L_i como un max HEAP

d) [1,0 punto] Indique las modificaciones a realizar en *getCristal(i, L, A)* para implementar su propuesta.

Puntaje	Pauta
0,5	A independiente del valor de i, extrae el elemento más prioritario del max HEAP A y lo alimenta al motor.
0,5	Extrae del max HEAP $L[i]$ el elemento más prioritario y lo inserta en el max HEAP A

Grafos Dijkstra

Una empresa desarrolla un sistema de navegación para una red de carreteras modelada como un grafo dirigido $G=(V,E)$, donde cada vértice representa una ciudad, y cada arista representa un camino de costo no negativo.

Actualmente utilizan el algoritmo de Dijkstra para calcular caminos de mínimo costo, sin embargo, el sistema solo necesita responder consultas del tipo: ¿Cuál es el camino de menor costo entre una ciudad origen s y una ciudad destino t ?

El algoritmo Dijkstra continúa ejecutándose hasta calcular la distancia de costo mínimo desde s hacia todas las ciudades del grafo, aún cuando únicamente interesa el camino hacia t .

Tomando en cuenta que Ud. puede usar el grafo transpuesto $G^T=(V,E^T)$, responda:

a) (1,5 pts.) Explique por qué ejecutar Dijkstra en un grafo grande (centenas o miles de V, E) puede realizar trabajo innecesario para responder las consultas del camino de menor costo de s a t . Proponga una modificación al algoritmo de Dijkstra (usando G^T) que permita terminar antes su ejecución cuando únicamente interesa conocer el camino mínimo entre s y t . Describa la modificación realizada y la condición bajo la cual el algoritmo termina;\

Solución:

(0,5 pts.) El algoritmo de Dijkstra calcula la distancia mínima desde un vértice origen \$\$\$ hacia todos los vértices del grafo. Cuando, para este problema, únicamente interesa responder consultas entre un origen \$\$\$ y un destino \$\$\$, gran parte del trabajo realizado por Dijkstra resulta innecesario.

Para reducir este trabajo puede utilizarse una búsqueda bidireccional:

- ejecutar Dijkstra desde el origen s sobre el grafo original;
- ejecutar simultáneamente Dijkstra desde el destino t sobre el grafo transpuesto G^T .

Sea μ el costo del mejor camino encontrado hasta el momento.

(0,5 pts.) El primer nodo donde se encuentran ambos caminos no necesariamente es el menor costo. El objetivo es que sea imposible que exista un camino más corto que el mejor encontrado hasta el momento. Entonces la condición de término es:

(0,5 pts.) $\mu \leq d_G + d_{GT}$ con d_G y d_{GT} son las distancias restantes en el grafo y su transpuesto.

donde $\mu = \min(\mu, d_G + d_{GT})$ las que están pendientes en las colas.

b) Modifique el pseudocódigo del algoritmo Dijkstra.\

Solución:

Básicamente en cada iteración se avanza en $d_G + d_{GT}$ simultáneamente.

(1,0 pt.) Inicialización de variables

$\text{distancias} \leftarrow \infty$, $\text{predecesores} \leftarrow \emptyset$, $\text{visitados} \leftarrow \text{false}$

inicialización de primeros nodos

$d_G[s] \leftarrow 0$

$d_{GT}[t] \leftarrow 0$

$Q_F \leftarrow \emptyset$, Cola desde s (hacia adelante)

$Q_B \leftarrow \emptyset$, Cola desde t (hacia atras)

$\mu \leftarrow \infty$

$\text{meet} \leftarrow \emptyset$ nodo de encuentro

(1,0 pt.) Ahora Dijkstra desde s y desde t integrado

While ($Q_F \neq \emptyset$ and $Q_B \neq \emptyset$)

bloque del while de dijkstra visto en clase desde inicio

bloque del while de dijkstra visto en clase desde gtermino

If ($\text{visit}_F[u]$ and ($\text{dist}_F[u] + \text{dist}_B[u] < \mu$))

$\mu \leftarrow \text{dist}_F[u] + \text{dist}_B[u]$

$\text{meet} \leftarrow u$

(1,0 pts.) condición de término los valores minimos en las colas mayor que el minimo μ

$$\text{If } \text{MinKey}(Q_F) + \text{MinKey}(Q_B) \geq \mu$$

c) (1,5 pts.) Justifique formalmente por qué la modificación propuesta siempre entrega el camino mínimo correcto. En particular, indique por qué el algoritmo puede detenerse en ese punto y no antes.

Solución:

Dijkstra garantiza que la distancia $s \rightarrow u$ es el camino más corto hacia adelante $t \rightarrow u$ es el camino mas corto hacia atras. Para cada u , μ es el mínimo entre $d_G + d_{GT}$

$d_G + d_{GT}$ son las menores distancias pendientes en cada búsqueda

Esta condición garantiza que cualquier camino que aún no se ha explorado tendrá un costo igual o mayor que μ . Por lo tanto, ya no es posible encontrar un camino más corto y el camino almacenado es el óptimo.

El algoritmo no puede detenerse antes porque, mientras $(d_f + d_b < \mu)$, todavía existe la posibilidad de descubrir un camino de menor costo.



Examen

7 de julio de 2026

1. Grafos

En un archipiélago del sur de Chile existen $n \geq 1$ islas, entre las que hay corrientes marinas unidireccionales que permiten viajar en balsa de una isla a otra sin costo de combustible (la corriente hace el trabajo), pero solo en la dirección de la corriente. Estas corrientes están dadas por un grafo dirigido $G = (V, E)$, donde V representa al conjunto de n islas del archipiélago y las aristas dirigidas representan las corrientes existentes. Dos islas se consideran en la misma **zona** si es posible viajar de una a la otra y también de vuelta usando solamente corrientes.

Se está organizando un festival que incluye a todo el archipiélago, con atracciones en cada isla. Por lo tanto, se quiere permitir que los habitantes puedan moverse de cualquier isla a cualquier otra. Las corrientes entre islas serán de gran ayuda, pero no necesariamente permiten conectar todo el archipiélago. Para suplir esta falta de conectividad, se decide establecer servicios de barcaza entre algunas islas.

Cada servicio de barcaza permite transportar personas en ambas direcciones entre dos islas y tiene un costo de implementación. Se han recibido m ofertas para establecer servicios de barcaza, las que se representan mediante el conjunto

$$E' = \{(u_i, v_i, c_i) : 1 \leq i \leq m\},$$

donde (u_i, v_i, c_i) indica que se puede establecer un servicio de barcaza entre las islas u_i y v_i , con $u_i, v_i \in V$, a un costo $c_i \in \mathbb{R}^+$.

El objetivo es escoger un conjunto de servicios de barcaza **de costo total mínimo** tal que, después de implementarlos, sea posible viajar entre cualquier par de islas usando corrientes y barcazas. Se garantiza que las ofertas en E' permiten conectar todas las zonas del archipiélago.

Diseñe un algoritmo que determine el costo mínimo necesario para lograr este objetivo, respondiendo a los siguientes puntos:

1. (1.0 pt.) Escriba pseudocódigo que permita calcular las zonas a partir del grafo dirigido G que tiene la información de las corrientes. Si es necesario, puede citar cualquier algoritmo visto en clases, pero debe indicar qué calcula dicho algoritmo.
2. (1.0 pt.) Construya un grafo auxiliar G' a partir de las zonas calculadas en el punto anterior y de los servicios de barcaza candidatos en E' . Debe especificar claramente cuáles son los vértices y las aristas de G' , y qué costos tienen dichas aristas.
3. (2.0 pts.) Usando el grafo auxiliar G' , diseñe un algoritmo para calcular el costo mínimo de implementación de servicios de barcaza.
4. (1.0 pt.) Justifique la correctitud de su algoritmo.
5. (1.0 pt.) Analice el tiempo de ejecución de su algoritmo en función de $|V| = n$, $|E|$ y $|E'| = m$.

Solución a los puntos 1–3. El algoritmo completo es el siguiente:

Algorithm 1 Costo mínimo para conectar las zonas del archipiélago

Require: Grafo dirigido $G = (V, E)$, conjunto de ofertas E'

Ensure: Costo mínimo para conectar todas las zonas

```

1:  $Z \leftarrow \text{CFC}(G)$   $\triangleright Z$  es el conjunto de representantes de las CFC  $\triangleright$  Además, cada vértice  $u \in V$  queda
   marcado con su representante  $u.rep$ 
2:  $E^{\text{CFC}} \leftarrow \emptyset$ 
   forall  $(u_i, v_i, c_i) \in E'$  do
        $u_i.rep \neq v_i.rep$ 
3:  $E^{\text{CFC}} \leftarrow E^{\text{CFC}} \cup \{(u_i.rep, v_i.rep, c_i)\}$ 
4:
5:
6:  $G' \leftarrow (Z, E^{\text{CFC}})$ 
7:  $T \leftarrow \text{KRUSKAL}(G')$ 
8: return  $\text{COSTO}(T)$ 
```

en donde:

Algorithm 2 $\text{COSTO}(T)$

Require: Árbol $T = (V_T, E_T)$ con aristas ponderadas

Ensure: Suma de los costos de las aristas de T

```

1:  $costo \leftarrow 0$  forall  $(u, v, c) \in E_T$  do
2:    $costo \leftarrow costo + c$ 
3:
4: return  $costo$ 
```

Tener en cuenta:

- La línea 1 del algoritmo corresponde al punto 1 de la pregunta.
- Las líneas 2 a la 8 corresponden al punto 2 de la pregunta.
- Las líneas 9 y 10 (y el algoritmo $\text{COSTO}(T)$) corresponden al punto 3 de la pregunta.

Solución al punto 4. Por definición, dos islas pertenecen a la misma zona si es posible viajar de una a la otra y viceversa usando únicamente corrientes. Por lo tanto, las zonas del archipiélago corresponden exactamente a las componentes fuertemente conexas del grafo dirigido G .

Una vez calculadas estas zonas, cada una de ellas puede verse como un único “supervértice”, ya que todas las islas dentro de una misma zona ya están mutuamente conectadas sin necesidad de agregar barcazas. Las ofertas de barcaza que unen dos islas de una misma zona no contribuyen a conectar zonas distintas y, por lo tanto, deben descartarse al construir el grafo auxiliar. Es incorrecto no descartarlas.

El grafo auxiliar G' tiene entonces un vértice por cada zona y una arista no dirigida entre dos zonas cuando existe una oferta en E' que conecta una isla de una zona con una isla de la otra. El costo de dicha arista es el costo de la oferta correspondiente. Así, cualquier conjunto de barcazas que permita viajar entre cualquier par de islas del archipiélago induce un subgrafo conexo de G' , y recíprocamente, cualquier subgrafo conexo de G' determina un conjunto de barcazas que conecta todas las zonas.

Por lo tanto, el problema original se reduce a conectar todos los vértices de G' con costo total mínimo. Esto es precisamente el problema del árbol cobertor mínimo. Como el algoritmo de Kruskal calcula un árbol cobertor mínimo de G' , el conjunto de barcazas devuelto por el algoritmo tiene costo total mínimo entre todas las soluciones factibles. En consecuencia, el algoritmo es correcto.

Solución al punto 5. La complejidad del algoritmo es el siguiente:

- Punto 1: Asumiendo una representación de listas de adyacencia, las componentes fuertemente conexas del grafo G se calculan en tiempo $O(|V| + |E|)$ usando el algoritmo de Kosaraju estudiado en clases, asumiendo una representación de listas de adyacencia para el grafo.
- Punto 2: la complejidad de las líneas 2-8 es $O(|E'|)$, asumiendo que se representará G' usando listas de adyacencia.
- Punto 3: la complejidad de las líneas 9 y 10 es $O(|E'| \lg |E'|) = O(|E'| \lg |V^{\text{CFC}}|)$, tal como se vio en clases.

La complejidad total es $O(|V| + |E| + |E'| \lg |V^{\text{CFC}}|)$.

Rúbrica de evaluación.

- **Punto 1 (1.0 pt.): cálculo de las zonas**
 - 1.0 pt.: Identifica correctamente que las zonas deben calcularse a partir de las componentes fuertemente conexas del grafo dirigido y propone pseudocódigo correcto, o bien cita correctamente un algoritmo visto en clases (por ejemplo, Kosaraju) indicando qué calcula.
 - 0.5 pts.: Reconoce correctamente la noción de zona, pero el pseudocódigo es incompleto, ambiguo o con errores menores.
 - 0.0 pts.: No reconoce que las zonas corresponden a clases de mutua alcanzabilidad en el grafo dirigido, o propone un algoritmo incorrecto.
- **Punto 2 (1.0 pt.): construcción del grafo auxiliar G'**
 - 1.0 pt.: Construye correctamente G' usando un vértice por zona/CFC, agrega aristas a partir de las ofertas en E' , y elimina correctamente las ofertas que unen islas de una misma zona.
 - 0.5 pts.: Construye la idea general de G' correctamente, pero comete un error menor (por ejemplo, no explicita bien los vértices, no filtra claramente las aristas internas a una zona, o describe de forma ambigua los costos).
 - 0.0 pts.: El grafo auxiliar está mal definido o no representa correctamente el problema.
- **Punto 3 (2.0 pts.): algoritmo para calcular el costo mínimo**
 - 2.0 pts.: Reconoce correctamente que sobre G' se debe calcular un árbol cobertor mínimo y propone un algoritmo correcto (Kruskal en nuestro caso) para obtener el costo mínimo.
 - 1.5 pts.: La idea general es correcta (usar un MST sobre G'), pero el pseudocódigo es incompleto o con errores menores.
 - 1.0 pt.: Construye correctamente G' y propone conectar sus vértices minimizando costo, pero no identifica claramente un algoritmo correcto o no explica bien cómo obtener el costo.
 - 0.5 pts.: Hay intuición de que se debe conectar las zonas con costo mínimo, pero la solución es incorrecta o demasiado incompleta.
 - 0.0 pts.: No propone un algoritmo pertinente.
- **Punto 4 (1.0 pt.): justificación de correctitud**
 - 1.0 pt.: Justifica correctamente que:
 1. las zonas corresponden a grupos de islas ya mutuamente alcanzables usando corrientes;
 2. las ofertas entre islas de una misma zona son irrelevantes para conectar zonas distintas y deben descartarse obligatoriamente para la correctitud de la respuesta;
 3. conectar las zonas al menor costo se reduce a encontrar un árbol cobertor mínimo en G' .
 - 0.5 pts.: La justificación contiene la idea principal, pero es incompleta o poco precisa.

- 0.0 pts.: No justifica la correctitud o la justificación es incorrecta.

■ **Punto 5 (1.0 pt.): análisis de complejidad**

- 1.0 pt.: Analiza correctamente las tres etapas principales:
 - cálculo de las CFC en $O(|V| + |E|)$;
 - construcción de G' en $O(|E'|)$;
 - Kruskal en $O(|E'| \lg |E'|)$, o equivalentemente $O(|E'| \lg |V^{\text{CFC}}|)$.

Además, combina correctamente estas cotas en una complejidad total.

- 0.5 pts.: El análisis general es correcto, pero con errores menores de notación o sin descomponer claramente por etapas.
- 0.0 pts.: El análisis de complejidad es incorrecto o está ausente.